

Testing de Aplicaciones Spring Boot con Testcontainers

Fuente: [JetBrains Blog](#) – Siva Katamreddy, diciembre 2024

¿Por qué Testcontainers?

Las pruebas de integración tradicionales tienen tres problemas comunes:

- **Entornos compartidos:** un cambio en el servidor afecta a todos los desarrolladores.
- **Mocks o bases de datos en memoria (H2):** no reproducen el comportamiento real de producción.
- **Configuración manual:** frágil, inconsistente y difícil de reproducir.

[Testcontainers para Java](#) resuelve esto levantando contenedores Docker reales y desechables durante la ejecución de las pruebas. Usa Docker como runtime, se integra con JUnit 5 y TestNG, y garantiza aislamiento total entre ejecuciones.

Paso 1 – Agregar la dependencia base

Testcontainers ofrece una API genérica ([GenericContainer](#)) para cualquier imagen Docker.

Gradle:

```
testImplementation 'org.testcontainers:testcontainers:1.20.4'
```

Maven:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers</artifactId>
  <version>1.20.4</version>
  <scope>test</scope>
</dependency>
```

Uso básico con [GenericContainer](#) (ejemplo Redis)

```
GenericContainer<?> container = new GenericContainer<>("redis:7")
    .withExposedPorts(6379);

container.start();

String host = container.getHost();
int hostPort = container.getMappedPort(6379); // Puerto aleatorio en el host
```

```
System.out.println("Redis en: " + host + ":" + hostPort);

container.stop();
```

Punto clave: Testcontainers mapea el puerto del contenedor a un puerto aleatorio disponible en el host, evitando conflictos. Aunque no llames `stop()`, el contenedor se destruye al terminar la JVM gracias al sidecar [Moby Ryuk](#).

Paso 2 – Usar el módulo específico para PostgreSQL

Los módulos específicos de tecnología ofrecen métodos de conveniencia (JDBC URL, credenciales, health checks ya configurados).

Gradle:

```
testImplementation 'org.testcontainers:postgresql:1.20.4'
```

Maven:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <version>1.20.4</version>
  <scope>test</scope>
</dependency>
```

Uso de `PostgreSQLContainer`

```
PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17");
postgres.start();

String jdbcUrl      = postgres.getJdbcUrl();
String username     = postgres.getUsername();
String password     = postgres.getPassword();
String dbName       = postgres.getDatabaseName();

postgres.stop();
```

El módulo abstrae el número de puerto, las credenciales por defecto y las verificaciones de disponibilidad (readiness check). No necesitas conocer los detalles internos del contenedor.

Paso 3 – Integrar con JUnit 5

Opción A – Callbacks manuales (@BeforeAll / @AfterAll)

```
class TestcontainersWithJUnit5Callbacks {  
  
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>  
    ("postgres:17");  
  
    @BeforeAll  
    static void beforeAll() {  
        postgres.start();  
        // Configurar datasource de la app para apuntar al contenedor  
    }  
  
    @AfterAll  
    static void afterAll() {  
        postgres.stop();  
    }  
  
    @Test  
    void test1() { /* usa postgres */ }  
  
    @Test  
    void test2() { /* usa postgres */ }  
}
```

Opción B – Extensión JUnit Jupiter (recomendada)

Primero, agrega la dependencia:

Gradle:

```
testImplementation 'org.testcontainers:junit-jupiter:1.20.4'
```

Maven:

```
<dependency>  
  <groupId>org.testcontainers</groupId>  
  <artifactId>junit-jupiter</artifactId>  
  <version>1.20.4</version>  
  <scope>test</scope>  
</dependency>
```

Luego, usa las anotaciones:

```
@Testcontainers // Activa la extensión JUnit  
class TestcontainersWithJupiterExtension {
```

```
@Container // JUnit gestiona el ciclo de vida
static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>
("postgres:17");

@Test
void test1() { /* usa postgres */ }

@Test
void test2() { /* usa postgres */ }
}
```

Nota sobre `static`: Al declarar el campo como `static`, se crea **una sola instancia** del contenedor para toda la clase. Si lo declaras como campo de instancia (no `static`), se crea un contenedor **por cada método de prueba** — costoso en tiempo. Lo estático es la práctica estándar.

Paso 4 – Clonar el proyecto de ejemplo

El blog usa la aplicación `bookmarks` (Spring Boot + Spring Data JPA + PostgreSQL + Flyway).

```
git clone --branch flyway https://github.com/sivaprasadreddy/bookmarks.git
git checkout -b testcontainers
```

Agrega las dependencias de Testcontainers al proyecto:

Gradle:

```
testImplementation 'org.testcontainers:postgresql'
testImplementation 'org.testcontainers:junit-jupiter'
```

Maven:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
```

Spring Boot gestiona las versiones mediante su BOM (`spring-boot-dependencies`), por eso no se especifica versión en el proyecto `bookmarks`.

Paso 5 – Probar el repositorio con `@DataJpaTest`

El problema del H2 en pruebas

Usar H2 para probar código que corre con PostgreSQL en producción introduce inconsistencias reales. Por ejemplo:

```
-- Válido en PostgreSQL:  
INSERT INTO items(id, code, name) VALUES(?,?,?) ON CONFLICT DO NOTHING;  
  
-- Falla en H2 con error de sintaxis:  
-- "Syntax error in SQL statement ... ON[*] CONFLICT DO NOTHING"
```

La solución es usar PostgreSQL real en las pruebas también.

Versión con `@DynamicPropertySource`

```
@DataJpaTest  
@Testcontainers  
class BookmarkRepositoryTest {  
  
    @Container  
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>  
("postgres:17");  
  
    @DynamicPropertySource  
    static void configureProperties(DynamicPropertyRegistry registry) {  
        registry.add("spring.datasource.url", postgres::getJdbcUrl);  
        registry.add("spring.datasource.username", postgres::getUsername);  
        registry.add("spring.datasource.password", postgres::getPassword);  
    }  
  
    @Autowired  
    BookmarkRepository bookmarkRepository;  
  
    @Test  
    void shouldFindBookmarkById() {  
        var bookmark = new Bookmark("JetBrains Blog",  
"https://blog.jetbrains.com");  
        Long id = bookmarkRepository.save(bookmark).getId();  
  
        var result = bookmarkRepository.findBookmarkById(id).orElseThrow();  
  
        assertThat(result.getTitle()).isEqualTo("JetBrains Blog");  
        assertThat(result.getUrl()).isEqualTo("https://blog.jetbrains.com");  
    }  
}
```

`@DynamicPropertySource` inyecta las propiedades del datasource **después** de que el contenedor arranca, sobrescribiendo las del `application.properties`.

Paso 6 – Simplificar con `@ServiceConnection` (Spring Boot 3.1+)

Spring Boot 3.1 introdujo soporte nativo para Testcontainers a través de `@ServiceConnection`. Agrega la dependencia:

Gradle:

```
testImplementation 'org.springframework.boot:spring-boot-testcontainers'
```

Maven:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
```

Prueba simplificada

```
@DataJpaTest
@Testcontainers
class BookmarkRepositoryTest {

    @Container
    @ServiceConnection // Spring Boot autoconfigura el
    datasource
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:17");

    @Autowired
    BookmarkRepository bookmarkRepository;

    @Test
    void shouldFindBookmarkById() {
        // mismo test, sin @DynamicPropertySource
    }
}
```

`@ServiceConnection` detecta que el contenedor es PostgreSQL y configura automáticamente `spring.datasource.*`. No necesitas declarar las propiedades manualmente.

Paso 7 – Centralizar la configuración con TestcontainersConfiguration

En lugar de repetir la declaración del contenedor en cada clase de prueba, centralízala:

```
// src/test/java/com/jetbrains/bookmarks/TestcontainersConfiguration.java

@TestConfiguration(proxyBeanMethods = false)
class TestcontainersConfiguration {

    @Bean
    @ServiceConnection
    PostgreSQLContainer<?> postgresContainer() {
        return new PostgreSQLContainer<>(DockeImageName.parse("postgres:17"));
    }
}
```

Spring Boot genera esta clase automáticamente cuando creas un proyecto nuevo con las dependencias PostgreSQL + Testcontainers. En proyectos existentes debes crearla manualmente.

Uso en las pruebas con @Import

```
@DataJpaTest
@Import(TestcontainersConfiguration.class) // Importa el contenedor centralizado
class BookmarkRepositoryTest {

    @Autowired
    BookmarkRepository bookmarkRepository;

    @Test
    void shouldFindBookmarkById() {
        // test sin ninguna declaración de contenedor
    }
}
```

Paso 8 – Prueba de integración completa del REST API

Para verificar el comportamiento end-to-end, usa @SpringBootTest que carga el contexto completo de la aplicación.

```
@SpringBootTest(webEnvironment = RANDOM_PORT) // Levanta servidor en puerto aleatorio
@Import(TestcontainersConfiguration.class)
class BookmarkControllerTest {

    @Autowired
```

```

private TestRestTemplate restTemplate;

@Autowired
private BookmarkRepository bookmarkRepository;

@BeforeEach
void setUp() {
    bookmarkRepository.deleteAllInBatch(); // Estado limpio por prueba
}

@Test
void shouldGetAllBookmarks() {
    bookmarkRepository.save(new Bookmark("JetBrains Blog",
    "https://blog.jetbrains.com"));
    bookmarkRepository.save(new Bookmark("IntelliJ IDEA
Blog","https://blog.jetbrains.com/idea/"));

    Bookmark[] bookmarks = restTemplate.getForObject("/api/bookmarks",
Bookmark[].class);

    assertThat(bookmarks.length).isEqualTo(2);
    assertThat(bookmarks[0].getTitle()).isEqualTo("IntelliJ IDEA Blog"); //
más reciente primero
    assertThat(bookmarks[1].getTitle()).isEqualTo("JetBrains Blog");
}
}

```

El endpoint `GET /api/bookmarks` devuelve los bookmarks ordenados del más nuevo al más antiguo. La prueba valida ese orden explícitamente.

Resumen del flujo completo

```

Proyecto Spring Boot
|
|— src/test/java/
|   |— TestcontainersConfiguration.java    ← Contenedor centralizado
|   (@ServiceConnection)
|   |
|   |— BookmarkRepositoryTest.java        ← @DataJpaTest +
|   @Import(TestcontainersConfiguration)
|       Prueba solo la capa de persistencia (JPA slice)
|   |
|   |— BookmarkControllerTest.java        ← @SpringBootTest +
|   @Import(TestcontainersConfiguration)
|       Prueba el sistema completo (REST → Service → Repository → PostgreSQL real)

```

Decisiones de diseño importantes

Decisión	Por qué importa
Usar PostgreSQL real en pruebas, no H2	Evita falsos negativos por diferencias de dialecto SQL
<code>@Container static</code> en lugar de instancia	Un contenedor por clase, no por método — más rápido
<code>@ServiceConnection</code> en lugar de <code>@DynamicPropertySource</code>	Menos código, más mantenible (Spring Boot 3.1+)
<code>TestcontainersConfiguration</code> centralizada	No repites la declaración del contenedor en cada clase
<code>@BeforeEach deleteAllInBatch()</code>	Aislamiento de datos entre pruebas dentro de la clase

Recursos

- [Código completo del ejemplo](#)
- [Módulos disponibles en Testcontainers](#)
- [ServiceConnection en Spring Boot](#)
- [Runtimes soportados](#)